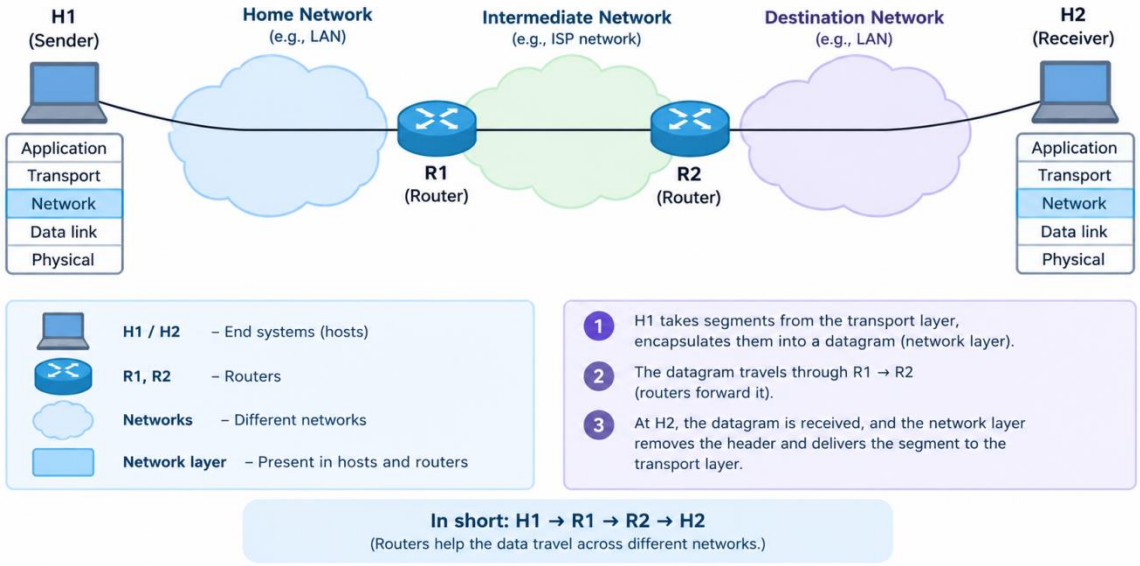
MODULE 3: NETWORK LAYER

NETWORK LAYER

The **network layer** is responsible for **moving packets from a sending host to a receiving host** through one or more routers.

Consider a network where **H1 sends data to H2 through routers R1, R2, etc.**

- At **H1**, the network layer takes **segments from the transport layer** and encapsulates them into **datagrams (network-layer packets)**.
- The datagrams are sent to the nearby router **R1**.
- The **routers forward datagrams** from their input links to the appropriate output links.
- At **H2**, the network layer receives the datagrams from **R2**, extracts the transport-layer segments, and delivers them to the transport layer.
- Routers mainly perform **network-layer forwarding** and normally do not run application or transport-layer protocols.



FORWARDING AND ROUTING

The main role of the **network layer** is to move packets from a **sending host to a receiving host**. It performs two important functions: **forwarding** and **routing**.

1. Forwarding

Forwarding is a **router-local action**. It means moving a packet from the router's **input interface to the appropriate output interface**.

For example:
H1 → R1 → R2 → H2
When a packet reaches **R1**, R1 checks the packet header and decides which output interface should be used to send it toward R2.

2. Routing

Routing is a **network-wide process**. It determines the **complete path** that packets should follow from the source to the destination.

The algorithms used to determine these paths are called **routing algorithms**, such as **Link-State (LS)** and **Distance-Vector (DV)** algorithms.

Difference between Forwarding and Routing

Forwarding	Routing
Router-local action	Network-wide process
Moves packet from input to output interface	Determines the complete path
Happens at each router	Determines the route from source to destination
Uses forwarding table	Uses routing algorithms

Routing decides the path; forwarding moves the packet along the path.

Forwarding Table

Every router has a **forwarding table**. It is used to decide **where an arriving packet should be sent next**.

The process is:
Packet arrives → Router examines header → Looks up forwarding table → Selects output interface → Forwards packet

For example, if the packet has header value **0111**, the forwarding table may show:
0111 → Interface 2

Therefore, the router sends the packet through **interface 2**.

The header value may be the **destination address** or information identifying the connection, depending on the network-layer protocol.

How is the Forwarding Table Created?

The **routing algorithm determines the information inserted into the forwarding table**. Routing protocol messages are used to configure the table.

So:
Routing algorithm → Forwarding table → Forwarding
This shows the relationship between routing and forwarding.

Packet Switch and Router

A **packet switch** is a general device that transfers a packet from an input interface to an output interface.

- **Link-layer switch:** makes forwarding decisions using **Layer 2** information.
- **Router:** makes forwarding decisions using **Layer 3 (network-layer)** information.

Therefore, a **router is a Layer 3 device**, although it also uses Layer 2 protocols.

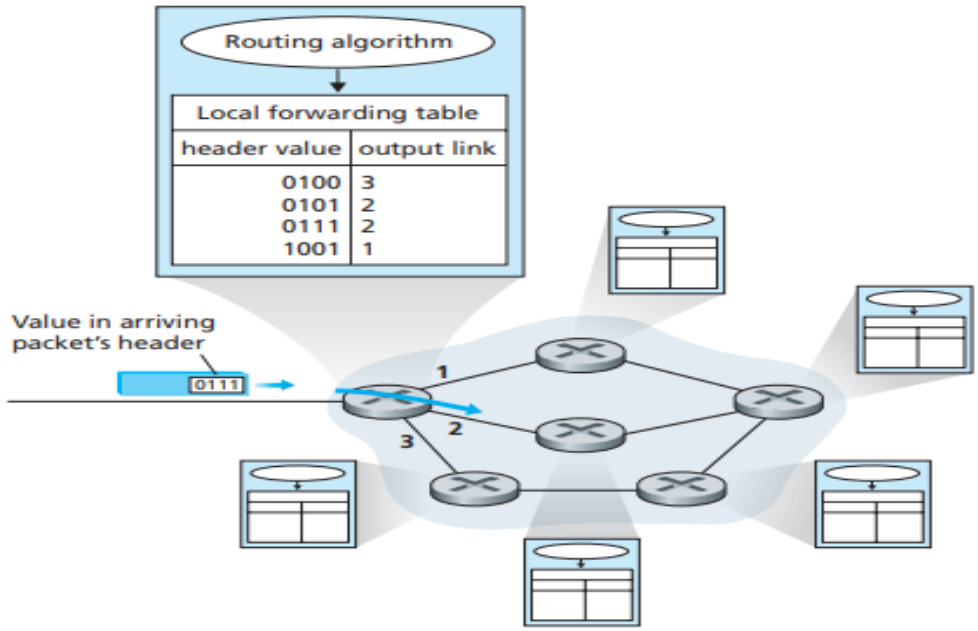


Figure 4.2 ♦ Routing algorithms determine values in forwarding tables

Connection Setup

The network layer has two main functions: **forwarding** and **routing**. In some networks, it also has a third function called **connection setup**.

Connection setup is the process of establishing the required **state information** before data packets can flow. Similar to the **TCP three-way handshake**, the routers along the selected path **handshake with each other and establish state** before data transmission.

For example, if the selected path is:

H1 → R1 → R2 → H2

R1 and R2 establish the necessary state before packets flow through the path.

Examples: ATM, Frame Relay, and MPLS use connection setup.

In short:
Routing → selects the path → Connection setup → establishes state → Data transfer

NETWORK SERVICE MODELS

A **network-layer service model** defines the characteristics of **end-to-end packet delivery** between the sending and receiving hosts.

It describes what the network layer **can guarantee** when delivering packets.

Possible Network-Layer Services

The network layer could provide different types of services:

1. Guaranteed Delivery

Guarantees that a packet will **eventually reach its destination**.

2. Guaranteed Delivery with Bounded Delay
Guarantees that the packet reaches the destination **within a specified time**, for example, within **100 ms**.

3. In-Order Packet Delivery
Guarantees that packets arrive at the destination in the **same order in which they were sent**.

4. Guaranteed Minimal Bandwidth
Guarantees a **minimum transmission rate**, such as **1 Mbps**, between the sender and receiver.

5. Guaranteed Maximum Jitter
Guarantees that the **time gap between successive packets** at the sender is approximately maintained at the receiver.

6. Security Services
The network layer can provide:

- **Confidentiality** – protects data using encryption.
- **Data integrity** – ensures data is not modified.
- **Source authentication** – verifies the source of the data.

Internet Best-Effort Service
The Internet network layer provides **best-effort service**. Here, **"best effort" does not mean that the service is guaranteed to be good**.

It means: **The network tries its best to deliver packets, but it does not guarantee successful delivery.**

Therefore, the Internet does **not guarantee**:

- Packet delivery
- Packet ordering
- Packet timing
- Bandwidth
- Congestion-free delivery

Packets may therefore be **lost, delayed, or delivered out of order**.

Example:

```
Sender → Packet 1 → ✓ Receiver
       → Packet 2 → ✓ Receiver
       → Packet 3 → ✗ Lost
```

The network has made a **best effort**, but it did not guarantee that Packet 3 would arrive.

Easy to remember:
Best effort = "I will try my best, but I cannot promise."

ATM Service Models
Unlike the Internet, **ATM provides different service models**. Two important ones are:

1. Constant Bit Rate (CBR)

CBR provides a **constant and guaranteed transmission rate**.

- Constant bandwidth is guaranteed.
- Cells are delivered **in order**.
- Cell delay and **jitter** are controlled within specified limits.
- Cell loss is controlled within specified limits.
- Suitable for **real-time audio and video**.

It is similar to having a **dedicated fixed-bandwidth link** between the sender and receiver.

2. Available Bit Rate (ABR)

ABR can be considered a **slightly better-than-best-effort service**.

- A **Minimum Cell Rate (MCR)** is guaranteed.
- Cells **may still be lost**.
- Cells are delivered **in order**.
- Timing is **not guaranteed**.
- The network can provide **congestion feedback** to the sender.
- If network resources are available, the sender may transmit at a rate higher than the MCR.

Comparison

Service	Bandwidth	Loss	Ordering	Timing	Congestion Indication
Internet Best Effort	No guarantee	No guarantee	Any order	Not maintained	None
ATM CBR	Constant rate guaranteed	Controlled within limits	In order	Maintained	Congestion will not occur
ATM ABR	Minimum rate guaranteed	May occur	In order	Not maintained	Provided

Virtual-Circuit and Datagram Networks

The network layer can provide either **connectionless service** or **connection service** between two hosts.

Connectionless vs Connection Service

- **Connectionless service:** No connection is established before data transfer.
- **Connection service:** A connection is established between the source and destination before data transfer. This involves **handshaking**.

This is similar to transport-layer services, but there is an important difference:

Transport Layer	Network Layer
Provides service **process-to-process**	Provides service **host-to-host**
Connection setup occurs mainly at the **end systems**	Connection setup involves **routers also**
Example: TCP connection	Example: Virtual Circuit

In major network architectures, the network layer generally provides either:

- **Virtual-Circuit (VC) network** → connection service
- **Datagram network** → connectionless service

Virtual-Circuit Networks

A **Virtual Circuit (VC)** is a logical connection established between a source and destination through a particular path. The Internet is a **datagram network**, while **ATM and Frame Relay** are examples of VC networks.

A VC consists of:

1. **A path** – series of links and routers between source and destination.
2. **VC numbers** – a VC number is assigned to each link along the path.
3. **Forwarding-table entries** – each router along the path stores information about the VC.

Example

Suppose the path is:

```
Host A → R1 → R2 → Host B
```

Suppose VC numbers are:

```
A ──12──> R1 ──22──> R2 ──32──> B
```

The packet carries a **VC number in its header**.When the packet reaches R1, R1 checks its forwarding table and **replaces VC number 12 with 22**.At R2, **22 is replaced with 32**.Thus, the VC number can be different on each link.

Why are different VC numbers used?

- It reduces the size of the VC field in the packet header.
- More importantly, each link can choose its VC number independently, making **VC setup simpler**.

Connection State in Routers

In a VC network, routers must maintain **connection state information** for every ongoing VC.

When a new VC is established:

- An entry is added to the router's forwarding table.

When the VC is terminated:

- The corresponding entry is removed.

This is an important difference from a connectionless datagram network.

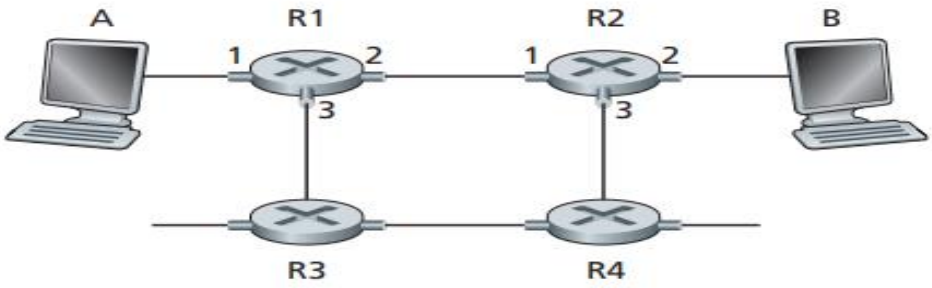


Figure 4.3 ◆ A simple virtual circuit network

Incoming Interface	Incoming VC #	Outgoing Interface	Outgoing VC #
1	12	2	22
2	63	1	18
3	7	2	17
1	97	3	87
...	...	...	...

Three Phases of a Virtual Circuit

1. VC Setup

Before data transfer:

- The sender requests a VC to the destination.
- The network determines the **path**.
- VC numbers are assigned to the links.
- Forwarding-table entries are created in the routers.
- The network may also **reserve resources**, such as bandwidth.

2. Data Transfer

After the VC is established, packets can **flow along the selected path**.

```
Host A → R1 → R2 → Host B
```

3. VC Teardown

When communication is finished:

- The sender or receiver requests termination.
- The VC is removed.
- The corresponding forwarding-table entries in the routers are deleted.

VC Setup vs TCP Connection Setup

This is an important distinction.

TCP connection setup:

- Involves only the **two end systems**.
- Intermediate routers are **not aware of the TCP connection**.

VC network setup:

- Involves the **end systems and routers along the path**.

- Each router maintains information about the VC passing through it.

Signaling Messages

The messages used to:

- establish a VC,
- modify router connection state, and
- terminate a VC

are called **signaling messages**.

The protocols used to exchange these messages are called **signaling protocols**.

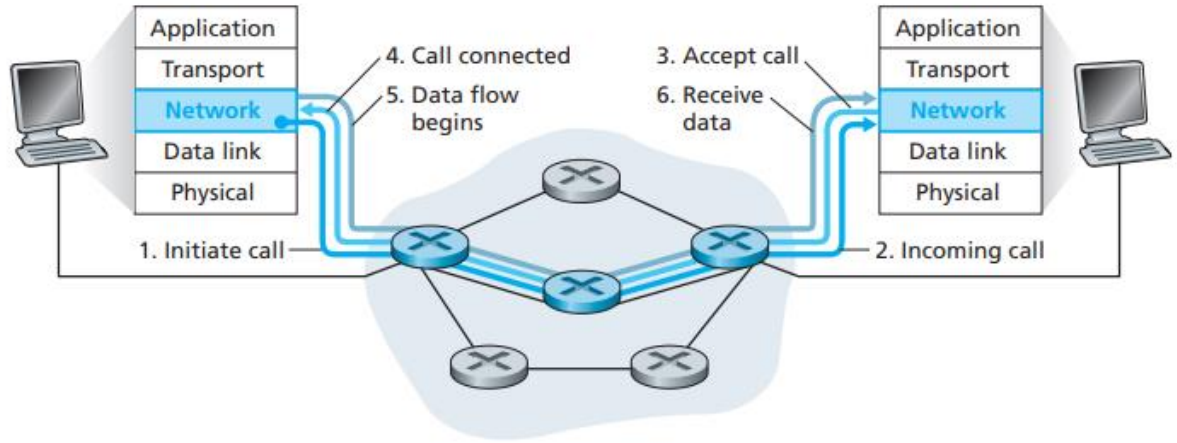


Figure 4.4 ♦ Virtual-circuit setup

DATAGRAM NETWORKS

Datagram Networks

A **datagram network** provides a **connectionless network-layer service**.

When a source wants to send a packet:

- There is **no VC setup** before transmission.
- The source puts the **destination address** in the packet header.
- The packet is sent into the network.
- Routers do **not maintain VC connection state**, because there is no virtual circuit.

How a Datagram Network Works

A packet passes through a series of routers:

```
Source → R1 → R2 → R3 → Destination
```

Each router independently decides where to forward the packet.

The router:

1. Examines the **destination address** in the packet.
2. Looks up the address in its **forwarding table**.
3. Finds the appropriate **outgoing link interface**.
4. Forwards the packet through that interface.

Prefix Matching in Forwarding Tables

A router does not need to maintain one forwarding-table entry for **every possible destination address**.

For example, an IPv4 destination address has **32 bits**, giving more than **4 billion possible addresses**. Maintaining billions of separate entries would be impractical.

Instead, routers use **prefixes**.

For example:

Prefix Match	Link Interface
`11001000 00010111 00010`	0
`11001000 00010111 00011000`	1
`11001000 00010111 00011`	2
Otherwise	3

The router compares the **beginning (prefix)** of the destination address with the entries in the forwarding table.

Example

Suppose the destination address is:

```
11001000 00010111 00010110 10100001
```

The beginning of this address matches:

```
11001000 00010111 00010
```

Therefore, the router forwards the packet to:

Link interface 0

So, instead of storing the complete address of every destination, the router can use **prefixes to represent a range of destination addresses**.

Longest Prefix Matching

Sometimes a destination address can match **more than one prefix** in the forwarding table.

For example, consider:

```
Destination:
11001000 00010111 00011000 10101010
```

This address matches both:

```
11001000 00010111 00011000
```

and

```
11001000 00010111 00011
```

There are therefore **multiple matches**.

In this situation, the router uses the **longest prefix matching rule**.

Longest Prefix Matching

When multiple forwarding-table entries match a destination address, the router selects the entry having the **longest matching prefix**.

In the above example:

- `11001000 00010111 00011000` → **24-bit prefix**
- `11001000 00010111 00011` → **21-bit prefix**

The **24-bit prefix is longer**, so the router chooses that entry.

Therefore, the packet is forwarded according to the interface associated with the **24-bit prefix**.

Why important?
Longest-prefix matching allows routers to make a **more specific forwarding decision** when multiple entries match.

Forwarding State in Datagram Networks

Although routers in a datagram network do not maintain **connection/VC state**, they still maintain **forwarding state** in their forwarding tables.

These forwarding tables are updated by **routing algorithms**.

In a datagram network, forwarding-table changes generally happen relatively slowly, typically around **every 1–5 minutes** in the textbook discussion.

In a VC network, the forwarding table can change whenever a VC is **established or terminated**.

Packets Can Follow Different Paths

Since forwarding tables can change, packets sent from the same source to the same destination **may not always follow the same path**.

For example:

```
Packet 1: Source → R1 → R2 → Destination
Packet 2: Source → R1 → R3 → Destination
```

Therefore, packets may:

- Follow **different paths**
- Arrive at the destination **out of order**

Key Points

Datagram network:

No VC setup → Destination address in packet → Prefix matching → Longest-prefix matching → Forward through appropriate interface

What's Inside a Router?

A router performs **forwarding**, which means transferring a packet from an incoming link to the appropriate outgoing link. A router has **four main components**.

1. Input Ports

- Receive packets from the **incoming link**.
- Perform **physical-layer and data-link-layer** functions.
- Examine the packet header and consult the **forwarding table**.
- Determine the appropriate **output port** for the packet.

2. Switching Fabric

- Connects the **input ports to the output ports**.
- Transfers the packet internally from the selected input port to the required output port.
- It can be considered as a **network inside the router**.

3. Output Ports

- Receive packets from the switching fabric.
- **Store/buffer** packets when necessary.
- Perform data-link and physical-layer functions.
- Transmit the packet through the **outgoing link**.

4. Routing Processor

- Runs **routing protocols**.
- Maintains routing information.
- Calculates and updates the **forwarding table**.

- Also performs **network management** functions.

Forwarding Plane and Control Plane

Forwarding Plane (Data Plane):

- Input ports + switching fabric + output ports.
- Performs the actual packet forwarding.
- Usually implemented in **hardware** for high-speed processing.

Control Plane:

- Mainly the routing processor.
- Performs routing, maintains routing information, and manages the router.
- Usually implemented in **software**.

Overall Working

Incoming Link → Input Port → Forwarding Table → Switching Fabric → Output Port → Outgoing Link

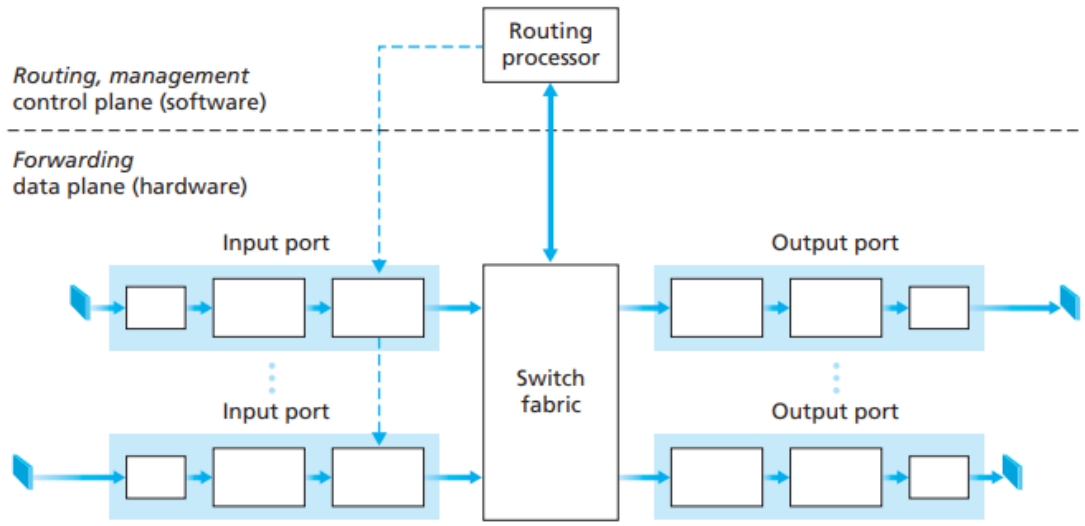


Figure 4.6 ♦ Router architecture

FUNCTIONING OF A ROUTER

Input Port Processing

When a packet arrives at a router, the **input port** performs these main steps:

1. **Line termination**
   - Receives the incoming signal from the physical link.
   - Handles the **physical layer**.
2. **Data-link processing**
   - Processes the incoming frame.
   - Removes the data-link header/trailer (**decapsulation**).
   - Passes the IP packet upward.
3. **Lookup**

- Checks the packet's **destination IP address**.
- Uses the **forwarding table** to determine the correct **output port**.
- Uses **longest prefix matching**.

4. **Forwarding**
   - Once the output port is identified, the packet is sent to the **switching fabric**.
5. **Queuing**
   - If the switching fabric is busy, the packet may temporarily wait in a **queue at the input port**.
6. **Other processing**
   - Router checks fields such as:
     - IP version
     - Checksum
     - TTL
   - Updates network-management counters.

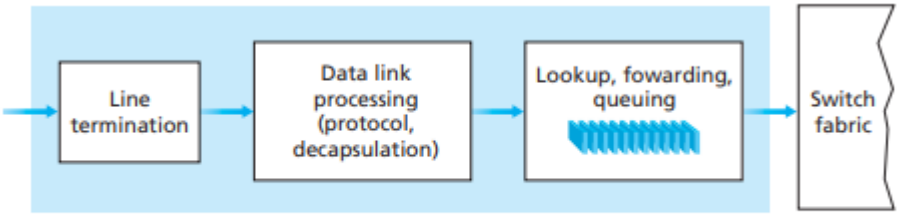


Figure 4.7 ♦ Input port processing

Switching Fabric

The switching fabric is the heart of a router. It transfers packets from an input port to the appropriate output port.

Three switching techniques:

1. **Switching via Memory**
   - **Packet moves through Input → Memory → Output.**
   - **The packet is stored in memory before being sent to the output port.**
   - **Limitation: memory access speed.**
2. **Switching via Bus**
   - **Input port sends the packet through a shared bus.**
   - **Only the destination output port accepts the packet.**
   - **Limitation: only one packet can cross the shared bus at a time.**
3. **Switching via Interconnection Network**
   - **Uses a crossbar switch to connect input and output ports.**
   - **Multiple packets can be forwarded in parallel.**
   - **If two packets require the same output port, one must wait.**

Easy comparison

Method	**Main idea**	**Limitation**
Memory	Input → Memory → Output	Memory speed
Bus	All packets use one shared bus	Bus bandwidth
Interconnection	Multiple paths between inputs & outputs	Same output may cause waiting

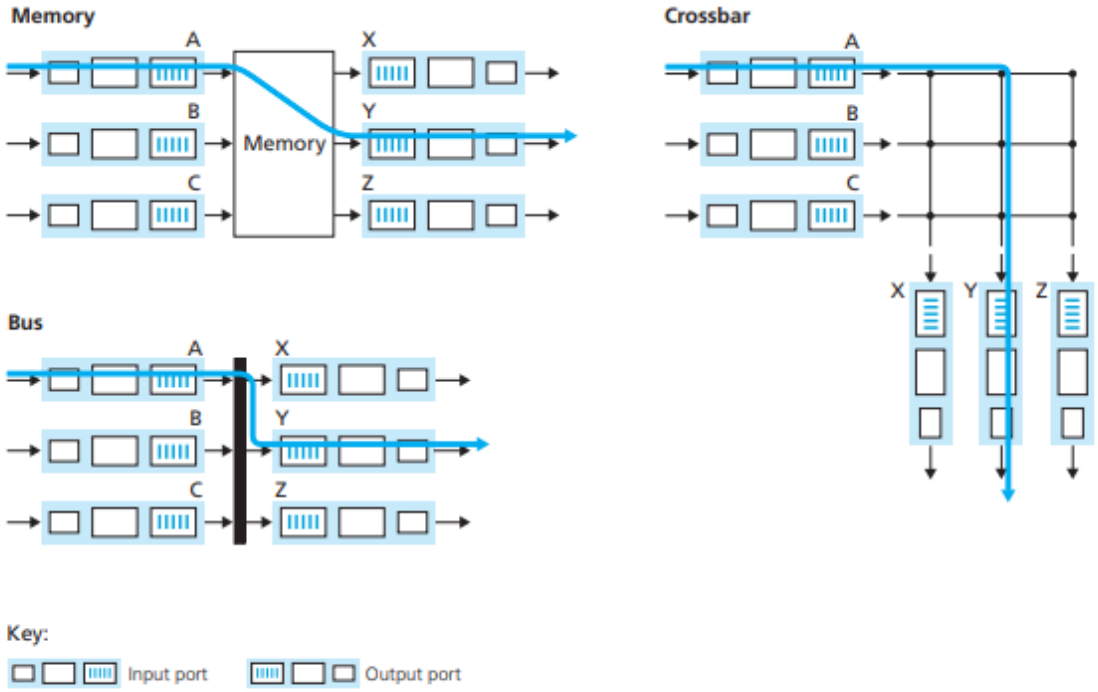


Figure 4.8 ♦ Three switching techniques

Output Port Processing

- Takes packets from the **output port's memory/queue** and sends them through the **output link**.
- **Selects and removes (de-queues)** packets for transmission.
- Performs **data-link layer and physical-layer processing** before transmission.

Flow:

Output queue → De-queuing → Data-link processing → Physical transmission

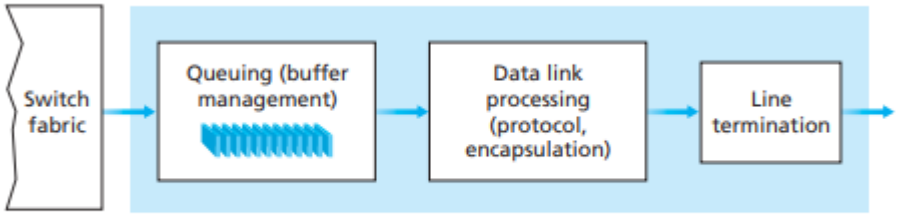


Figure 4.9 ♦ Output port processing

Router Queuing

- **Queues can form at both input ports and output ports** when packets cannot be forwarded/transmitted immediately.
- The amount of queuing depends on **traffic load, switching-fabric speed, and link speed**.
- If router **memory becomes full**, newly arriving packets are **dropped/lost**.

Input Port Queuing

- Occurs when packets arrive faster than the **switching fabric** can transfer them.
- If the switching fabric is sufficiently fast, input queuing is usually small.

Output Port Queuing

- Occurs when **multiple packets are destined for the same output port**.
- The output link can transmit only one packet at a time, so other packets must wait in the queue.

- If the queue becomes full, packets are **dropped**.

Packet Scheduling and Queuing

1. Packet Scheduling

- When multiple packets are waiting at an **output port**, a scheduler selects which packet to transmit.
- **FCFS:** First packet to arrive is transmitted first.
- **WFQ:** Shares the link fairly among different connections.
- Scheduling helps provide **Quality of Service (QoS)**.

2. Packet Dropping

- When the buffer is full, packets may be **dropped**.
- **Drop-tail:** Newly arriving packet is dropped when the buffer is full.
- **RED (Random Early Detection):** Packets may be dropped/marked **before the buffer becomes full** to signal congestion.

3. Input Queuing and HOL Blocking

- If the switching fabric is not fast enough, packets wait in **input queues**.

Head of the Line

- Two packets at the **front of input queues** want the same upper-right output.
- Only one can be transferred, so the other **must wait**.
- The packet behind it also waits, **even though its middle-right output is free**.
- This is called **Head-of-the-Line (HOL) blocking**.

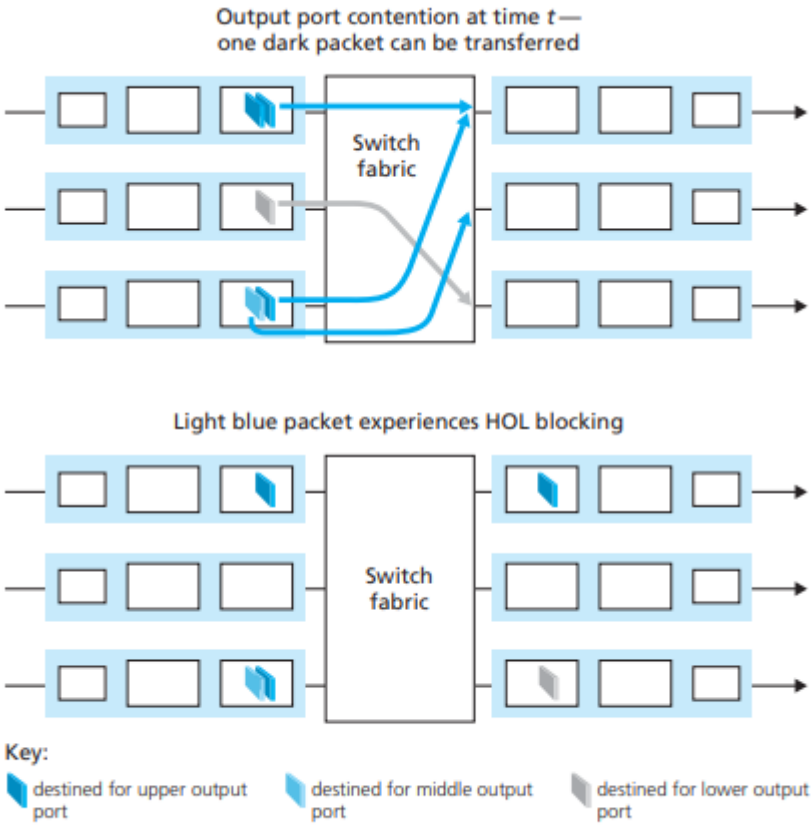


Figure 4.11 ♦ HOL blocking at an input queued switch

The Routing Control Plane

- The **routing control plane** is responsible for making **routing decisions** and maintaining routing information.

- Traditionally, it runs in the **routing processor of each router**.
- The control plane is **distributed**: different routers run routing algorithms and exchange control messages with each other.
- Newer architectures separate the **control plane** from the **data plane**:
    - **Data plane:** remains in the router and performs packet forwarding.
    - **Control plane:** can be partly or fully implemented in an **external centralized server**.
- This separation can **simplify routing** and allow different control systems to use the same fast hardware.

INTERNET PROTOCOL

• IP (Internet Protocol): Handles addressing and forwarding of datagrams. The two versions are IPv4 and IPv6.
• Routing: Determines the path of datagrams from source to destination and creates forwarding tables.
• ICMP (Internet Control Message Protocol): Reports network-layer errors and provides network information.

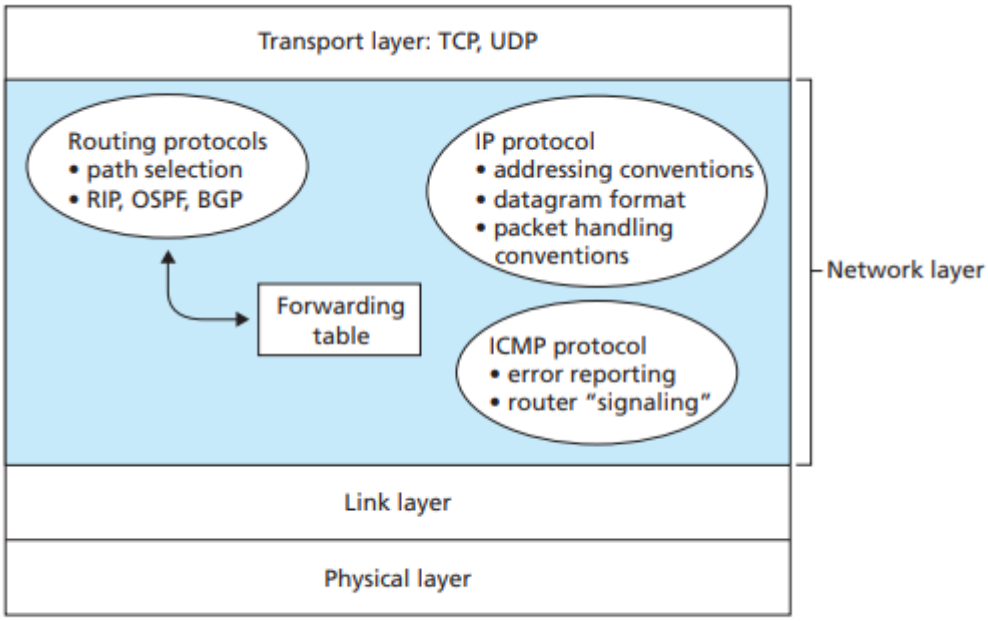


Figure 4.12 ♦ A look inside the Internet's network layer

IPv4 Datagram Format

A network-layer packet is called a datagram. An IPv4 datagram consists of a header and a data (payload) field. The header contains information required for addressing, forwarding, and handling the datagram.

Important Fields of an IPv4 Datagram

1. Version Number

- A 4-bit field that specifies the IP version.
- For IPv4, the value is 4.
- Routers use this field to determine how to interpret the remaining fields of the datagram.
- IPv4 and IPv6 use different datagram formats.

2. Header Length

- Specifies the length of the IPv4 header.
- The header length can vary because the header may contain optional fields.
- The usual header length is 20 bytes when there are no options.

3. Type of Service (ToS)

- Helps distinguish different types of IP datagrams according to their service requirements.
- These requirements may include low delay, high throughput, or reliability.
- Example: Real-time voice traffic requires low delay, whereas FTP traffic may not have the same requirement.
- The level of service provided depends on the policy configured by the router administrator.

4. Datagram Length

- Specifies the total length of the datagram, including both header and data, measured in bytes.
- This field is 16 bits long, so the maximum theoretical datagram size is 65,535 bytes.
- However, datagrams are rarely larger than 1,500 bytes.

5. Identifier, Flags and Fragmentation Offset

These three fields are used in IP fragmentation, which occurs when a datagram must be divided into smaller fragments to pass through a network that supports a smaller maximum packet size.

- Identifier: Helps identify fragments belonging to the same original datagram.
- Flags: Control fragmentation. For example, a flag can indicate that fragmentation is not allowed or that more fragments follow.
- Fragmentation Offset: Indicates the position of a fragment's data within the original datagram, helping the destination reassemble the fragments.

In IPv6, routers do not fragment packets.

6. Time-to-Live (TTL)

- Prevents datagrams from circulating indefinitely due to routing loops.
- Each router decreases the TTL value by 1.
- When TTL reaches 0, the datagram is discarded.

7. Protocol

- Identifies the transport-layer protocol to which the datagram's data must be delivered at the destination.
- Examples:
  - TCP — protocol number 6
  - UDP — protocol number 17

The protocol field connects the network layer and transport layer. It serves a role similar to port numbers, which connect the transport layer and application layer.

8. Header Checksum

- Used to detect errors in the IPv4 header.
- The checksum is calculated by treating each 2 bytes of the header as a number and adding them using one's-complement arithmetic. The one's complement of the sum is stored in the checksum field.
- The receiving router calculates the checksum again and compares it with the received checksum. If an error is detected, the datagram is typically discarded.
- The checksum must be recalculated at each router because fields such as TTL change during forwarding.

Why are checksums used at both the network and transport layers?

- The IP checksum covers only the IP header.
- TCP/UDP checksums cover their respective transport-layer segments.
- IP and TCP/UDP may also operate with different underlying protocols, so both layers need their own error-checking mechanisms.

9. Source IP Address

- Contains the IP address of the host that originally sent the datagram.
- The source host inserts its own IP address into this field.

10. Destination IP Address

- Contains the IP address of the final destination host.
- The source host often obtains the destination IP address through a DNS lookup.

11. Options

- Allows additional information or special processing instructions to be included in the IPv4 header.
- Options are rarely used to avoid unnecessary overhead in ordinary datagrams.
- Options make the header variable in length and can increase the processing time required by routers.
- For these and other reasons, the IPv6 basic header does not include an options field; additional information is handled through extension headers.

12. Data (Payload)

- Contains the actual information being transported.
- Usually, the data field contains a TCP or UDP segment.
- It may also carry other information, such as ICMP messages.

IP Datagram Fragmentation

IP fragmentation is the process of dividing a large IP datagram into smaller datagrams called fragments so that they can pass through a link with a smaller MTU.

1. Why is Fragmentation Needed?

- MTU (Maximum Transmission Unit): The maximum size of the data that a link-layer frame can carry.
- Different links have different MTUs. For example, Ethernet has an MTU of 1,500 bytes.
- If a datagram is larger than the outgoing link's MTU, the router divides it into smaller fragments.

2. Fragmentation and Reassembly

- Fragmentation: The router divides the original datagram into smaller fragments.
- Reassembly: The destination host combines the fragments to reconstruct the original datagram.
- Reassembly is done at the destination, not at routers, to keep routers simple.
- If any fragment is missing, the incomplete datagram is discarded. TCP can recover by retransmitting the lost data.

3. Fields Used in Fragmentation

1. Identification: All fragments of the same original datagram have the same identification number.
2. Flags: The More Fragments (MF) flag is 1 if more fragments follow and 0 for the last fragment.
3. Fragmentation Offset: Indicates the position of the fragment's data in the original datagram. It is measured in units of 8 bytes.

Example

A 4,000-byte datagram (20-byte header + 3,980-byte data) passes through a link with an MTU of 1,500 bytes.

Fragment	Data size	Offset	MF flag
1st	1,480 bytes	0	1
2nd	1,480 bytes	185	1
3rd	1,020 bytes	370	0

Each fragment has its own 20-byte IP header. The maximum data size of the first two fragments is 1,480 bytes (1,500 − 20). Fragment data sizes, except for the last fragment, must be multiples of 8 bytes.

Offsets: 1480/8=185 and 2960/8=370

Disadvantages

- Increases complexity at routers and destination hosts.
- Loss of any fragment prevents reconstruction of the original datagram.
- Malformed or overlapping fragments may cause DoS attacks.

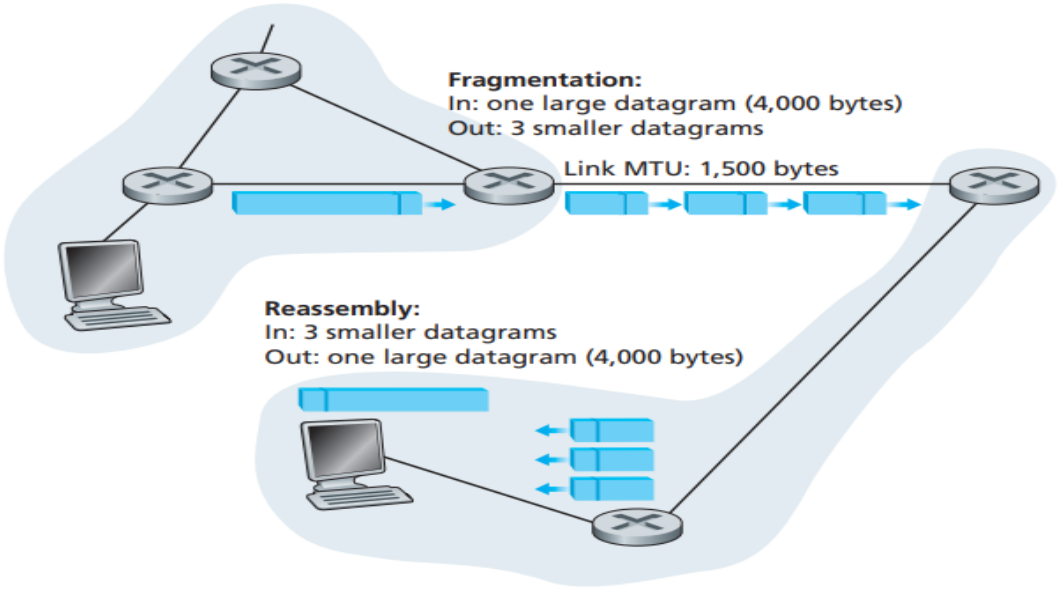


Figure 4.14 ♦ IP fragmentation and reassembly

IPV4 ADDRESSING

Before discussing IPv4 addressing, we must understand how hosts and routers connect to the network.

- **Host:** A host typically has a single link to the network. When IP wants to send a datagram, it sends it through this link.
- **Interface:** The boundary between a host or router and its physical link is called an interface.

Interface Address

- A host, such as a computer, usually has one or two interfaces, such as Ethernet and Wi-Fi.
- A router has multiple interfaces because it connects to different networks. Each interface has its own IP address.
- Each IP address is 32 bits (4 bytes) long in IPv4.

Example of an IPv4 address:

```
223.1.1.1
```

It is represented in dotted-decimal notation, with four decimal numbers separated by dots. Each number represents 8 bits and ranges from 0 to 255.

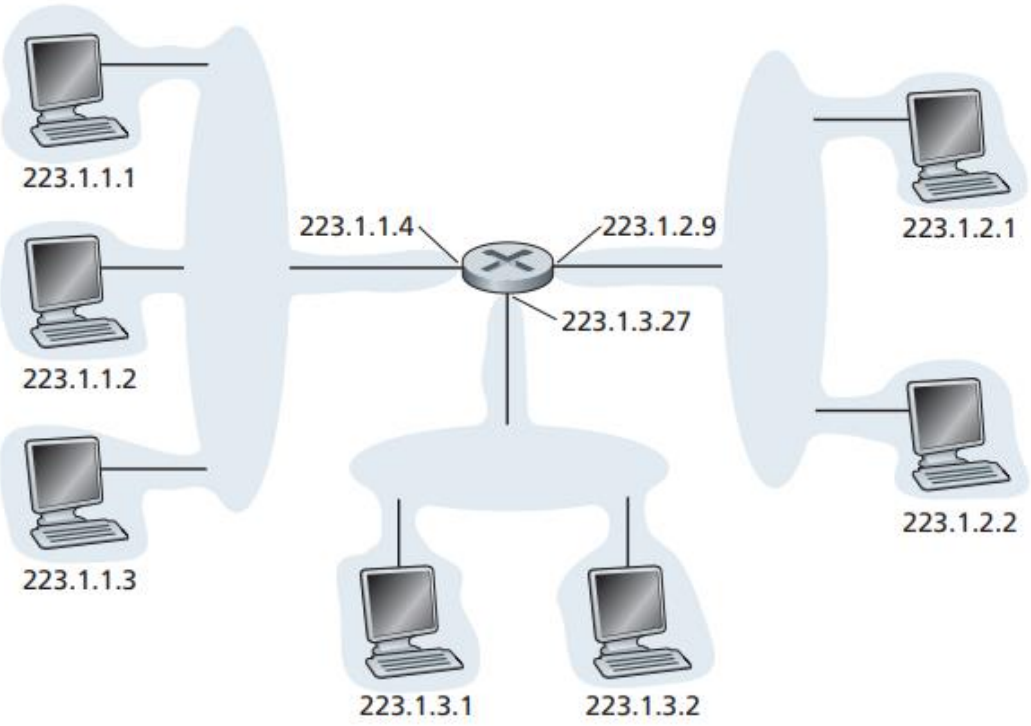


Figure 4.15 ♦ Interface addresses and subnets

- **Router:** A router receives a datagram through one link and forwards it through another link. Therefore, a router has two or more links and multiple interfaces, one for each link.
- **IP Address:** Every host and router interface can send and receive IP datagrams. Hence, each interface must have its own IP address.

IPV4 Addressing

- **IP address length:** An IPv4 address is 32 bits (4 bytes) long, giving 2^{32} approximately 4 billion possible IP addresses.
- **Dotted-decimal notation:** An IP address is written as four decimal numbers separated by dots. Each number represents 8 bits (one byte).

Example: 193.32.216.9
Binary form: 11000001 00100000 11011000 00001001

- **Globally unique address:** Every host and router interface on the global Internet must have a globally unique IP address, except interfaces behind NAT.
- **Subnet:** A subnet is a network connecting interfaces without an intervening router. It is also called an IP network or network.
- **Subnet address:** The subnet address identifies the subnet. For example, 223.1.1.0/24, where /24 means the leftmost 24 bits represent the subnet address.
- Example: The subnet 223.1.1.0/24 contains three host interfaces (223.1.1.1, 223.1.1.2, 223.1.1.3) and one router interface (223.1.1.4). Other hosts in this subnet must have addresses of the form 223.1.1.xxx.

- **Other subnets:** In the example, 223.1.2.0/24 and 223.1.3.0/24 are two additional subnets.
- **Point-to-point links:** When routers are interconnected using point-to-point links, each link connecting two router interfaces forms a separate subnet. For example, the links between routers R1–R2, R2–R3, and R3–R1 form the subnets 223.1.9.0/24, 223.1.8.0/24, and 223.1.7.0/24, respectively.

Subnets

- To identify subnets, detach each interface from its host or router. Each isolated network formed is called a subnet.
- In Figure 4.17, this method gives six subnets.
- An organization with multiple Ethernet segments and point-to-point links has multiple subnets. Devices within the same subnet share the same subnet address.
- Different subnets may have different addresses, but their addresses often share common bits.

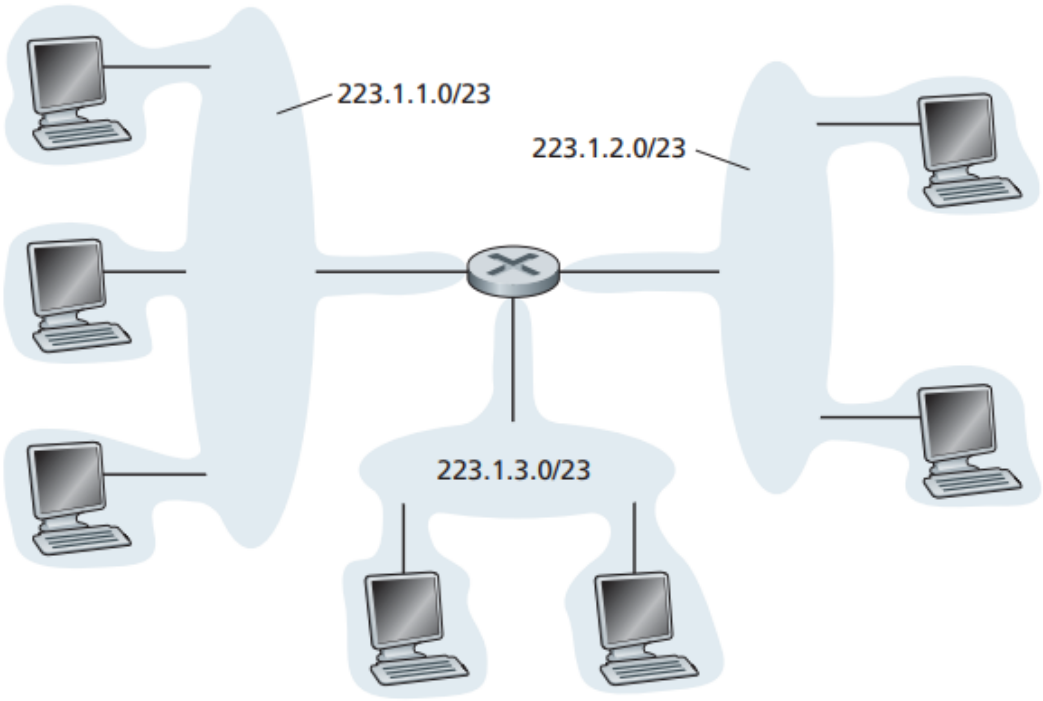


Figure 4.16 ♦ Subnet addresses

In figure 4.17

- Each subnet connects multiple host interfaces or router interfaces within the same network.
- The three routers connect these separate subnets and allow hosts in different subnets to communicate.
- Each router has multiple interfaces, and each interface has its own IP address.
- Each subnet has a unique subnet address.
- The six subnets include the local networks connected to hosts and the networks connecting the routers.

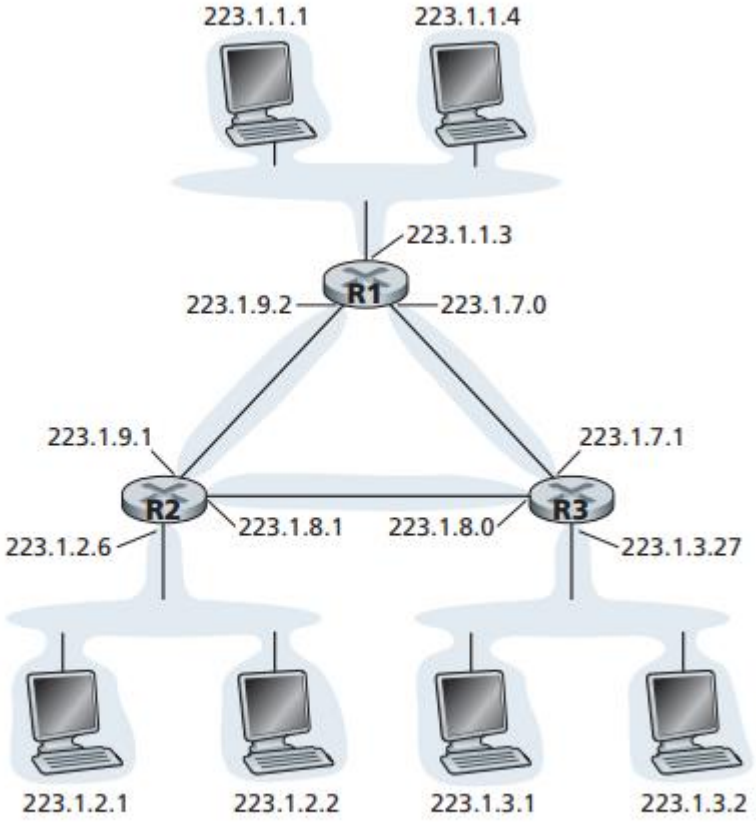


Figure 4.17 ♦ Three routers interconnecting six subnets

NETMASK

A subnet mask indicates how many leftmost bits of a 32-bit IP address define the subnet address.

- A subnet consists of host interfaces and router interfaces connected to the same network.
- The notation /24, sometimes called a subnet mask , indicates that the leftmost 24 bits define the subnet address.
- The remaining 8 bits identify the interfaces within that subnet.
- All interfaces belonging to the same subnet have addresses with the same subnet prefix.

Example:

Subnet address: 223.1.1.0/24

Interface	IP address
Host 1	223.1.1.1
Host 2	223.1.1.2
Host 3	223.1.1.3
Router interface	223.1.1.4

Here, /24 indicates that the first 24 bits represent the subnet address. Any additional host connected to this subnet must have an address beginning with 223.1.1.

The equivalent subnet mask in dotted-decimal notation is 255.255.255.0.

CIDR (Classless Interdomain Routing)

- CIDR is the Internet's IP address assignment strategy.
- It divides a 32-bit IP address into two parts: the network portion and the host portion.

- CIDR uses the format a.b.c.d/x, where x indicates the number of leading bits used for the network portion (prefix).
- **Network prefix:** The \(x\) most significant bits (leftmost bits) of an IP address in the form a.b.c.d/x represent the network portion, called the prefix.
- **Address block:** An organization is usually assigned a block of consecutive IP addresses that share a common prefix.
- **Address aggregation:** An ISP can advertise a single network prefix representing multiple organizations' networks. This is also called route aggregation or route summarization.
- Example: An ISP advertises 200.23.16.0/20 to represent address blocks belonging to eight organizations. This reduces the number of routing table entries needed by other routers.
- **Longest prefix matching:** If two routes match a destination, routers choose the route with the longest matching prefix (the most specific route).
  Example: If the Internet receives these two routes:
- 200.23.16.0/20 — advertised by Fly-By-Night-ISP.
- 200.23.18.0/23 — advertised by ISPs-R-Us for Organization 1.
  Traffic destined for Organization 1 follows the /23 route because it has the longer, more specific prefix.

Advantages: Routers outside an organization need to consider only the network prefix, reducing the size of their forwarding tables. Internal routers use the remaining bits to identify devices and subnets within the organization.

- In CIDR, the first \(x\) bits represent the network prefix, shared by all devices in an organization.
- The remaining \(32-x\) bits identify individual hosts or subnets within the organization.
- Internal routers use these remaining bits to forward packets to the correct devices or subnets.
- Example: In a /21 address, the first 21 bits represent the network prefix, and the remaining 11 bits identify hosts or internal subnets. A /24 prefix can identify a specific subnet within the organization.

Classful Addressing

- Before CIDR, IP addresses used fixed network prefix lengths: 8 bits (Class A), 16 bits (Class B), and 24 bits (Class C).
- Class C (/24): Supports 254 usable host addresses, which may be too small for some organizations.
- Class B (/16): Supports 65,534 usable host addresses, which may be too large.
- Problem: An organization with 2,000 hosts would typically receive a Class B block, leaving over 63,000 addresses unused.
- This caused IP address wastage and rapid depletion of Class B addresses. CIDR was introduced to allow more flexible address allocation.

IP Broadcast Address

- 255.255.255.255 is the IP broadcast address.
- A datagram sent to this address is delivered to all hosts on the same subnet.
- Routers usually do not forward these messages to other subnets.

IP Address Assignment

IP addresses are assigned in two steps:

1. An organization receives a block of IP addresses.
2. Individual devices receive addresses from that block.

ICMP (Internet Control Message Protocol)

ICMP is a network-layer protocol specified in RFC 792 (Request for Comments 792). It is used by hosts and routers to communicate network-layer information. Its most common use is error reporting.

The Internet network layer has three main components: IP, Internet routing protocols (RIP, OSPF and BGP), and ICMP.

1. Functions of ICMP

- **Error reporting:** Reports problems when a packet cannot reach its destination. For example, during a Telnet, FTP or HTTP session, a router may send a "Destination network unreachable" message if it cannot find a path to the destination.
- **Network testing:** ICMP is used by Ping and Traceroute to check connectivity and identify the route between computers.
- **Router information:** ICMP also supports router advertisement and router discovery messages.

2. Working of ICMP

- ICMP is often considered part of IP, but it lies just above IP in the network architecture.
- ICMP messages are carried inside IP datagrams as payload, similar to TCP segments and UDP segments.
- When a host receives an IP datagram containing ICMP, it passes the contents to ICMP for processing.
- Each ICMP message contains a type and a code field that identify the message.
- An ICMP error message also contains the IP header and the first 8 bytes of the original IP datagram that caused the error. This helps the sender identify the packet responsible for the problem.

3. Important ICMP Message Types

The following are selected ICMP message types from Figure 4.23 in the textbook.

Type	Code	Description
0	0	Echo reply (Ping)
3	0	Destination network unreachable
3	1	Destination host unreachable
3	2	Destination protocol unreachable
3	3	Destination port unreachable
3	6	Destination network unknown
3	7	Destination host unknown
4	0	Source quench (congestion control)
8	0	Echo request (Ping)
9	0	Router advertisement
10	0	Router discovery
11	0	TTL expired
12	0	IP header bad

4. Ping

Ping is a program used to check whether a destination host is reachable and to measure the round-trip time.

Working:

1. The source host sends an ICMP Type 8, Code 0 (Echo Request) message.
2. The destination host receives the request and sends an ICMP Type 0, Code 0 (Echo Reply) message.
3. The source uses the reply to check connectivity and measure the time taken for the message to travel to the destination and back.

   Important point: Most TCP/IP implementations support the Ping responder directly in the operating system. It does not need to run as a separate application process.

5. Source Quench

- Source Quench was designed to provide feedback about network congestion.
- When a router became congested, it could send a Source Quench message asking the host to reduce its transmission rate.
- This message is rarely used in practice because TCP has its own congestion-control mechanism at the transport layer.

6. Traceroute

Traceroute is a program used to identify the routers between a source and destination and measure the round-trip time to each router.

Working:

1. The source sends IP datagrams containing UDP segments with an unlikely destination port number.
2. The first datagram has a TTL (Time to Live) of 1, the second has a TTL of 2, the third has a TTL of 3, and so on.
3. When a datagram's TTL expires at a router, the router discards it and sends an ICMP Type 11, Code 0 (TTL Expired) message to the source.
4. The source records the router's IP address and calculates the round-trip time using a timer.
5. As the TTL increases, the datagrams reach routers farther along the path.
6. Eventually, a datagram reaches the destination. Because the UDP port is unlikely to be in use, the destination sends an ICMP Type 3, Code 3 (Destination Port Unreachable) message.
7. When the source receives this message, it knows that Traceroute can stop.

   Important points:

- Traceroute helps identify the routers between the source and destination.
- It also measures the round-trip time to each router.
- The standard Traceroute program sends three probe packets for each TTL value, so it usually displays three results for each router.

IPV6

- IPv6 (Internet Protocol version 6) was developed by the IETF (Internet Engineering Task Force) because IPv4 addresses were becoming insufficient.
- IPv6 provides a much larger address space and improves some IPv4 features.
- In 2011, IANA (Internet Assigned Numbers Authority) allocated its remaining IPv4 addresses to regional registries.
- IPv6 resulted from the IPng (Internet Protocol next generation) project. ST-2 (Internet Stream Protocol Version 2) was an earlier protocol, not a successor to IPv4.

IPV6 Datagram

1. Expanded Addressing: IPv6 increases IP address size from 32 bits to 128 bits, providing a much larger address space. It supports unicast, multicast, and anycast addresses. Anycast delivers a datagram to one host in a group, usually the nearest one.

2. Simplified Header: IPv6 uses a fixed 40-byte header. Some IPv4 fields are removed or made optional, allowing faster processing of datagrams.

3. Flow Labelling and Priority: IPv6 uses a 20-bit flow label to identify packets belonging to a particular flow that may need special handling, such as audio and video transmission. Its 8-bit traffic class field can assign priority to packets within a flow or to packets from particular applications.

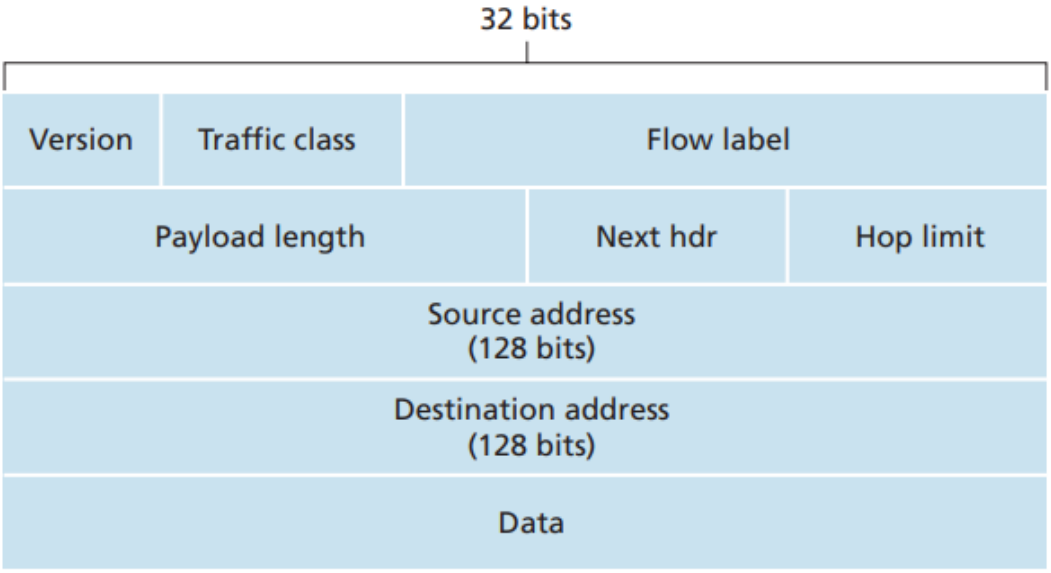


Figure 4.24 ♦ IPv6 datagram format

Fields are:

Version: This 4-bit field identifies the IP version number. Not surprisingly, IPv6 carries a value of 6 in this field. Note that putting a 4 in this field does not create a valid IPv4 datagram.

Traffic class: This 8-bit field is similar in spirit to the TOS field we saw in IPv4.

• **Flow label:** As discussed above, this 20-bit field is used to identify a flow of datagrams.

• **Payload length:** This 16-bit value is treated as an unsigned integer giving the number of bytes in the IPv6 datagram following the fixed-length, 40-byte datagram header.

• **Next header:** This field identifies the protocol to which the contents (data field) of this datagram will be delivered (for example, to TCP or UDP). The field uses the same values as the protocol field in the IPv4 header.

• **Hop limit:** The contents of this field are decremented by one by each router that forwards the datagram. If the hop limit count reaches zero, the datagram is discarded.

• **Source and destination addresses:** The various formats of the IPv6 128-bit address are described in RFC 4291.

• **Data:** This is the payload portion of the IPv6 datagram. When the datagram reaches its destination, the payload will be removed from the IP datagram and passed on to the protocol specified in the next header field.

Fields Removed from IPv4 in IPv6

The following IPv4 fields are removed or changed in IPv6 to simplify the header and improve packet processing speed.

1. Fragmentation and Reassembly: Routers do not fragment IPv6 datagrams. Only the source and destination handle fragmentation and reassembly. If a packet is too large, the router sends an ICMP Packet Too Big message to the sender.
2. Header Checksum: Removed because transport-layer protocols (TCP and UDP) and link-layer protocols (Ethernet) perform error checking. This avoids recalculating the checksum at every router.
3. Options: The options field is removed from the standard header. Optional information is handled using IPv6 extension headers.
Note: These are not all simply deleted fields: fragmentation is handled differently, and options are supported through extension headers.

Transition from IPv4 to IPv6
The Internet was originally based on IPv4. However, IPv4 and IPv6 are incompatible because IPv4-only devices cannot process IPv6 datagrams directly. Therefore, IPv6 must be introduced gradually while allowing existing IPv4 systems to continue working.
A flag day means upgrading all Internet devices to IPv6 at the same time. This is not practical because the Internet contains millions of networks and a huge number of devices.
According to RFC 4213, two important approaches are used to introduce IPv6 gradually: dual stacking and tunnelling.
1. Dual-Stack Approach
In the dual-stack approach, a device supports both IPv4 and IPv6. Such a device is called an IPv6/IPv4 node.

- It has both an IPv4 address and an IPv6 address and can send and receive both IPv4 and IPv6 datagrams.
- When communicating with an IPv4-only device, it uses IPv4. When communicating with an IPv6-capable device, it can use IPv6.
- The DNS (Domain Name System) helps the sender obtain the appropriate address information for the destination.
- If either the sender or receiver can communicate only through IPv4, IPv4 must be used for that communication.
  Limitation: When an IPv6 datagram must be converted into an IPv4 datagram, some IPv6-specific information, such as the flow label, cannot be preserved because IPv4 has no corresponding field.

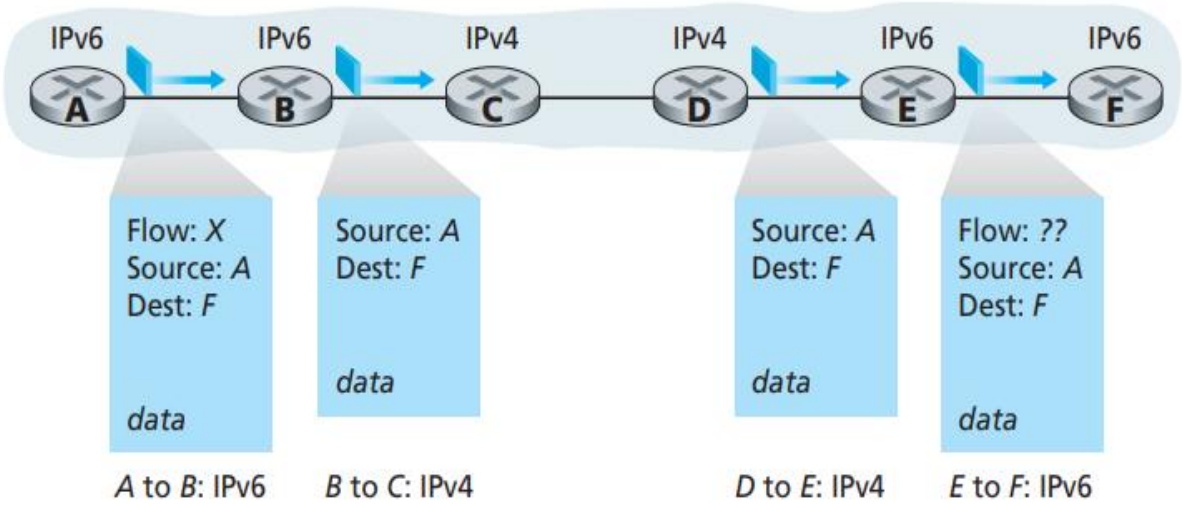


Figure 4.25 ♦ A dual-stack approach

2. Tunneling

Tunneling allows two IPv6 nodes to communicate through a network containing IPv4 routers. For example, suppose two IPv6 routers are connected through a network of IPv4 routers. Since the IPv4 routers cannot directly forward IPv6 datagrams, tunneling is used.

The process works as follows:

1. **Encapsulation:** The sending IPv6 node places the entire IPv6 datagram inside the data field of an IPv4 datagram.
2. **Transmission:** The IPv4 datagram travels through the IPv4 network. The intermediate IPv4 routers examine the IPv4 header and forward the datagram without processing the enclosed IPv6 datagram.
3. **Decapsulation:** At the receiving IPv6 node, the IPv4 header is removed, and the original IPv6 datagram is extracted.
4. **Forwarding:** The original IPv6 datagram is then processed and forwarded normally using IPv6.

   Advantage: Tunneling allows the original IPv6 datagram, including its IPv6-specific fields, to pass through an IPv4 network without converting the IPv6 datagram itself into an IPv4 datagram.

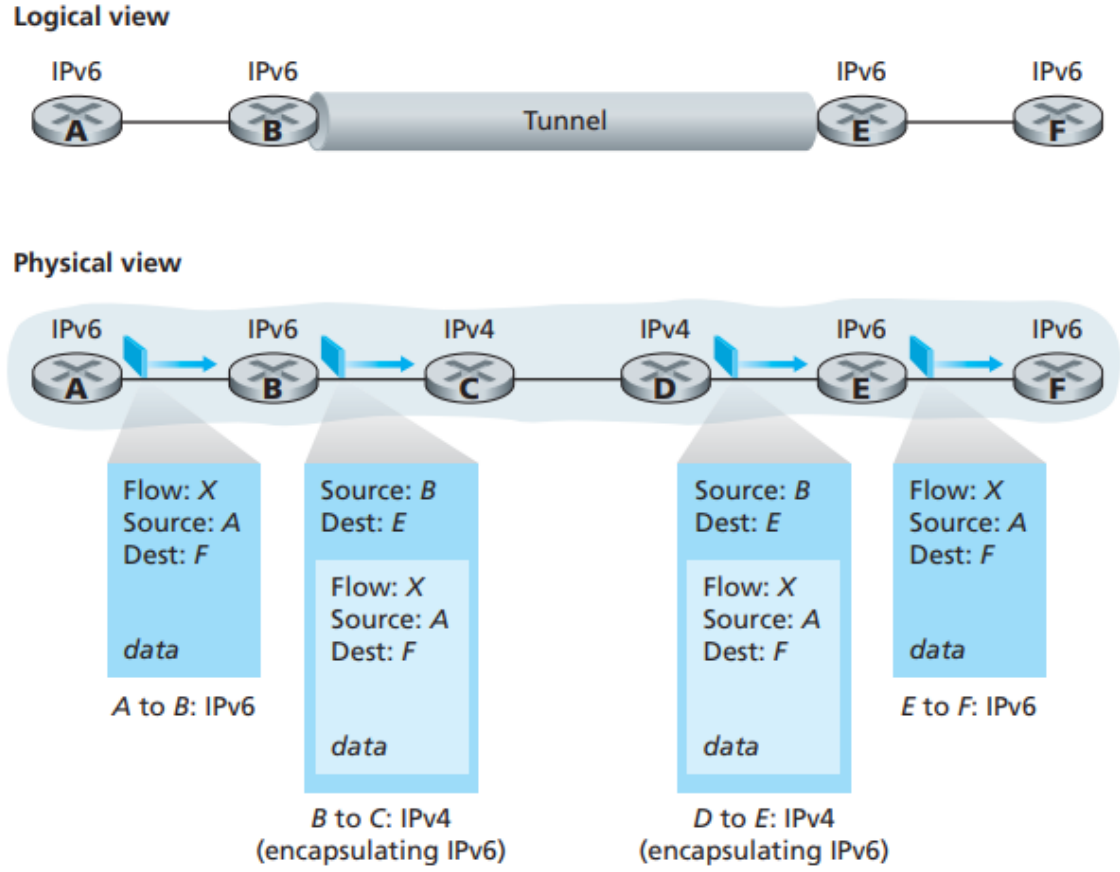


Figure 4.26 ♦ Tunneling

Adoption of IPv6

The adoption of IPv6 was initially slow because upgrading network devices and infrastructure takes time. However, its use gradually increased as the number of Internet-connected devices, including mobile phones, grew.

For example, 3GPP (3rd Generation Partnership Project) specified IPv6 as the standard addressing scheme for mobile multimedia. This helped encourage IPv6 adoption in mobile networks.

Challenges in Network-Layer Changes

Changing network-layer protocols is difficult because the changes may require updates to routers, hosts, and other network devices across many different networks. All these devices cannot be upgraded at the same time.

In contrast, application-layer protocols, such as the Web, instant messaging, peer-to-peer (P2P) file sharing, and audio/video streaming, are generally easier to introduce because they often require changes mainly to applications or software.

Therefore, changes at the network layer usually take place more slowly than changes at the application layer.

ROUTING ALGORITHM

1. Introduction to Routing

The network layer performs two important functions: forwarding and routing.

- **Forwarding:** When a packet arrives at a router, the router checks its forwarding table and determines the outgoing link interface through which the packet should be sent.
- **Routing:** Determines the path that packets take from the source to the destination through a network of routers.

  Routing algorithms exchange and calculate information used to configure forwarding tables.

  Whether the network layer provides a datagram service or a virtual-circuit (VC) service, it must determine the path packets take from source to destination. In a datagram service, different packets between the same source and destination may follow different paths. In a VC service, all packets belonging to the same virtual circuit follow the same path.

Default Router, Source Router and Destination Router

A host is usually connected directly to one router called its default router, also known as the first-hop router.

- **Default router (first-hop router):** The router to which a host sends its packets first.
- **Source router:** The default router of the source host.
- **Destination router:** The default router of the destination host.

Therefore, routing a packet from a source host to a destination host mainly involves finding a path from the source router to the destination router.

The purpose of a routing algorithm is to find a good path, usually the path with the least cost. In practice, routing decisions may also depend on policies. For example, an organization may prevent its router from forwarding packets originating from another organization's network.

2. Graph Model of a Network

A graph is used to represent a computer network and formulate routing problems.

A graph is represented as (G=(N,E)\), where:

- **Nodes (N):** Represent routers where packet-forwarding decisions are made.
- **Edges (E):** Represent the physical links connecting routers.
- **Link cost (c(x,y)):** Represents the cost of the link between routers (x) and (y).

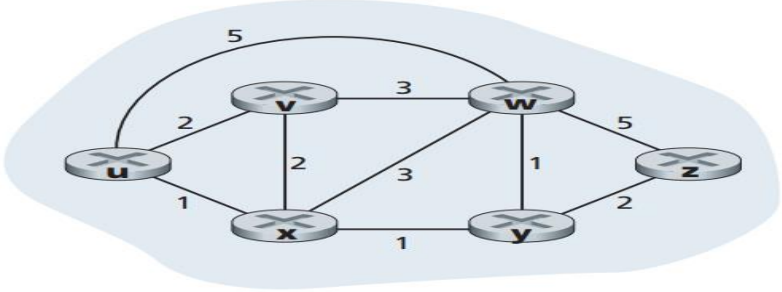


Figure 4.27 ◆ Abstract graph model of a computer network

Link cost may depend on physical distance, link speed, or monetary cost. For example, a transoceanic link may have a higher cost than a short terrestrial link.If no direct link exists between two nodes, its cost is considered infinity. In an undirected graph, a link can be used in either direction, so c(x,y)=c(y,x).A node (y) is called a neighbor of node (x) if a direct link connects them.

3. Least-Cost Path

A path is a sequence of connected nodes from a source to a destination, where each consecutive pair is connected by an edge.

The cost of a path is the sum of the costs of all links along that path.

A least-cost path is the path between the source and destination with the smallest total cost among all possible paths.

If all links have equal cost, the least-cost path is also the shortest path, which contains the fewest links.

4. Classification of Routing Algorithms

Routing algorithms can be classified in three main ways.

A. Global and Decentralized Routing Algorithms

1. Global Routing Algorithm (Link-State — LS)

- Has complete information about network connectivity and the cost of every link.
- Uses this global information to calculate least-cost paths.
- The calculation may be performed at one location or replicated at multiple routers.
- It is called a Link-State algorithm because it requires information about the cost of every link.

2. Decentralized Routing Algorithm (Distance-Vector — DV)

- No router has complete information about the costs of all network links.
- Each router initially knows only the costs of its directly connected links.
- Routers repeatedly exchange information with their neighboring routers and update their calculations.
- Through this iterative process, each router gradually estimates the least-cost paths to other destinations.

  It is called a Distance-Vector algorithm because each router maintains a vector of estimated costs (distances) to other nodes.

B. Static and Dynamic Routing Algorithms

1. Static Routing

- Routes change very slowly over time.
- Changes usually occur through human intervention, such as manually editing a router's forwarding table.

2. Dynamic Routing

- Routes change according to changes in network topology or link costs.
- The algorithm may run periodically or respond directly to network changes.
- Although dynamic routing adapts to changing conditions, it may cause routing loops or route oscillations, where routes change repeatedly.

C. Load-Sensitive and Load-Insensitive Routing Algorithms

1. Load-Sensitive Routing

- Link costs change dynamically according to the current congestion level.
- Congested links receive higher costs, encouraging the algorithm to select alternative routes.
- Early ARPAnet routing algorithms used this approach, but several difficulties were encountered.

2. Load-Insensitive Routing

- Link costs do not explicitly reflect the current or recent congestion level.
- Modern Internet routing algorithms, such as RIP (Routing Information Protocol), OSPF (Open Shortest Path First), and BGP (Border Gateway Protocol), are load-insensitive.

Link-State (LS) Algorithm

- The Link-State (LS) algorithm requires complete information about the network topology and the cost of every link.
- Each router broadcasts link-state packets containing the identities and costs of its directly connected links to other routers.
- As a result, all routers obtain an identical and complete view of the network.
- Each router independently runs Dijkstra's algorithm to calculate the least-cost paths from itself to all other nodes. OSPF (Open Shortest Path First) uses link-state routing.

Dijkstra's Algorithm

Dijkstra's algorithm finds the least-cost path from a source node to all other nodes in a network. It works step by step, selecting the node with the lowest path cost each time.
Important terms:

- D(v): cost of the least-cost path from the source node to destination v as of this iteration of the algorithm.
- p(v): previous node (neighbour of v) along the current least-cost path from the source to v.
- N': subset of nodes; v is in N' if the least-cost path from the source to v is definitively known.

Link-State (LS) Algorithm for Source Node u

Initialization:

```
N' = {u}
for all nodes v
  if v is a neighbor of u
    then D(v) = c(u,v)
  else D(v) = ∞
```

Loop

```
find w not in N' such that D(w) is a minimum
add w to N'
update D(v) for each neighbor v of w and not in N':
      D(v) = min( D(v), D(w) + c(w,v) )
/* new cost to v is either old cost to v or known
 least path cost to w plus cost from w to v */
```

until N'= N

Example

1. Initialization: The costs from source node u to its direct neighbours v, x, and w are D(v) = 2, D(x) = 1, and D(w) = 5. The costs to y and z are D(y) = ∞ and D(z) = ∞ because they are not directly connected to u.

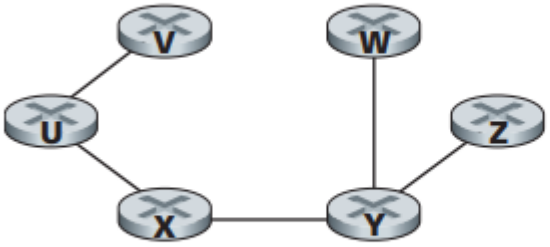
2. First iteration: Node x is selected because it has the lowest cost, D(x) = 1. Through x, the cost to w decreases from 5 to 4, so D(w) = 4 and p(w) = x. The cost to y becomes D(y) = 2 and p(y) = x. The cost to v remains D(v) = 2.
3. Second iteration: Nodes v and y both have the lowest cost, 2. In this example, y is selected. The set becomes N'. The costs to the remaining nodes are then updated.
4. Further iterations: The algorithm continues selecting the node with the lowest estimated cost and updating the costs of its neighbouring nodes until N' = N.
Result: The least-cost paths from source node u to all other nodes are determined.

step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

Table 4.3 ♦ Running the link-state algorithm on the network in Figure 4.27

Forwarding Table and Computational Complexity

- After Dijkstra's algorithm terminates, predecessor information is used to construct the least-cost paths from the source to all destinations.
- The forwarding table stores the next-hop node for each destination along its least-cost path.
- In the basic implementation, the algorithm searches the remaining nodes in each iteration to find the minimum cost.
- The total number of searches is n(n + 1)/2. Therefore, the worst-case computational complexity is $O(n^2)$.
- Using a heap data structure can reduce the computation time.



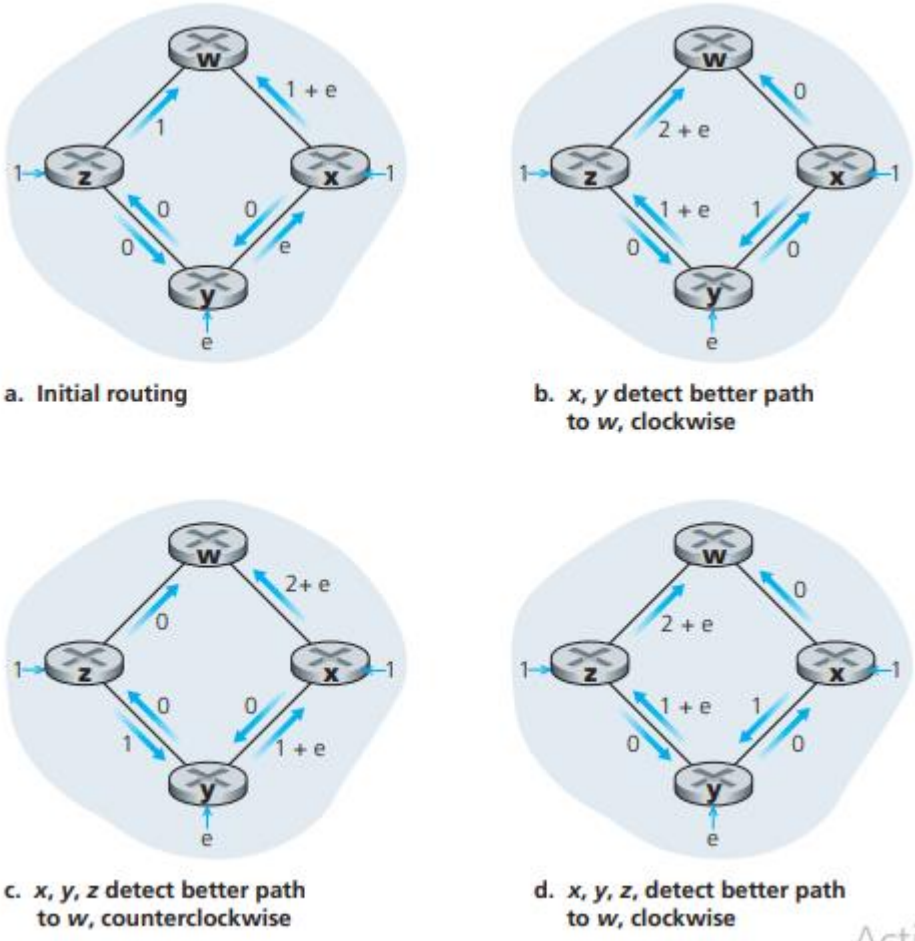
Destination	Link
v	(u, v)
w	(u, x)
x	(u, x)
y	(u, x)
z	(u, x)

Figure 4.28 ♦ Least cost path and forwarding table for nodule u

Route Oscillation in the LS Algorithm

Route oscillation occurs when link costs depend on the traffic load or delay on a link.
In Figure 4.29, nodes x and z send one unit of traffic each to destination w, while node y sends e units of traffic to w.

- **(a) Initial routing:** The nodes follow their initial routes, and the link costs depend on the traffic carried by each link.
- **(b) Clockwise route:** When the LS algorithm runs again, nodes x and y find that the clockwise path to w has a lower cost. They shift their traffic to this path, changing the link costs.
- **(c) Counterclockwise route:** In the next iteration, the counterclockwise path becomes cheaper. Nodes x, y, and z shift their traffic to this path.
- **(d) Clockwise route again:** The traffic changes the link costs once more, making the clockwise path cheaper. All three nodes switch back to the clockwise route.
  This process repeats because each change in traffic load affects link costs and causes routers to select a different path. This repeated switching is called route oscillation. It can occur in any routing algorithm that uses congestion- or delay-based link costs, not just LS.



a. Initial routing

b. *x, y* detect better path to *w*, clockwise

c. *x, y, z* detect better path to *w*, counterclockwise

d. *x, y, z,* detect better path to *w*, clockwise

Figure 4.29 ♦ Oscillations with congestion-sensitive routing

How to Prevent Route Oscillation

1. Make link costs independent of traffic load: This prevents traffic changes from affecting link costs, but makes it harder to avoid congested or high-delay links.
2. Randomize link-state advertisement times: Routers should not all run the LS algorithm at the same time. Randomizing when routers send link-state advertisements helps prevent them from becoming synchronized and reduces oscillations.

Distance Vector (DV) Routing Algorithm

1. Introduction

The Distance Vector (DV) algorithm is an iterative, asynchronous, and distributed routing algorithm. Unlike the Link State (LS) algorithm, which uses global network information, DV uses information received from directly connected neighboring routers.

- Distributed: Each node receives information from its neighbors, calculates the costs to destinations, and shares the results with its neighbors.
- Iterative: The process continues repeatedly until no more changes occur in the distance vectors.
- Asynchronous: Nodes do not need to perform calculations or exchange information at the same time.
- Self-terminating: The algorithm stops automatically when no further updates are required.

2. Bellman-Ford Equation

The Bellman-Ford equation is used to calculate the least-cost path from one node to another.

Let $d_x(y)$ be the cost of the least-cost path from node x to node y.

The Bellman-Ford equation is:

$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

Where:

- $d_x(y)$ = Least cost from node x to node y.
- $c(x,v)$ = Cost of the link between node x and its neighbor v.
- $d_v(y)$ = Least cost from neighbor v to destination y.
- $\min_v$ = Minimum value calculated over all neighbors of node x.

Explanation:

To find the least-cost path from node x to destination y, the node considers all its neighbors. It adds the cost of reaching each neighbor to that neighbor's cost of reaching the destination. The minimum of these values gives the least-cost path.

3. Example of the Bellman-Ford Equation

Consider node u, which has three neighbors: v, x, and w. The costs are given below.

- $c(u,v) = 2$
- $c(u,x) = 1$
- $c(u,w) = 5$

The costs from the neighboring nodes to destination z are:

- $d_v(z) = 5$
- $d_x(z) = 3$
- $d_w(z) = 3$

Using the Bellman-Ford equation:

$$d_u(z) = \min\{2+5,\ 1+3,\ 5+3\}$$

$$d_u(z) = \min\{7,\ 4,\ 8\} = 4$$

Therefore, the least cost from node u to node z is **4**, through neighbor x.

4. Importance of the Bellman-Ford Equation

The Bellman-Ford equation has two important uses:

1. Forwarding table: It helps a router identify the next-hop router for forwarding a packet along the least-cost path.
2. Information exchange: It provides the basis for communication between neighboring routers in the Distance Vector algorithm.

5. Working of the Distance Vector Algorithm

Each node \(x\) maintains a distance vector containing its estimated costs to reach all nodes in the network.

The distance vector is represented as:

$$D_x = [D_x(y) : y \in N]$$

Where:

- $D_x(y)$ = Estimated cost from node x to destination y.
- N = Set of all nodes in the network.

Each node maintains the following routing information:

1. **Link costs:** The cost c(x,v) of the link to each directly connected neighbor v.
2. **Its own distance vector:** The estimated cost from node x to every destination in the network.
3. **Neighbors' distance vectors:** The distance vectors received from all directly connected neighbors.

The algorithm works as follows:

1. Each node initially estimates the cost of reaching every destination.
2. From time to time, each node sends a copy of its distance vector to its neighbors.
3. When node x receives a new distance vector from neighbor v, it stores the information.
4. Node x updates its distance vector using the Bellman-Ford equation:

$$D_x(y) = \min_v\{c(x,v) + D_v(y)\}$$

5. If the distance vector changes, node x sends its updated vector to all its neighbors.
6. The neighboring nodes update their own distance vectors and share any changes with their neighbors.
7. This process continues until the distance vectors converge to the actual least-cost paths.

Distance-Vector (DV) Algorithm

At each node, x:

```
1  Initialization:
2     for all destinations y in N:
3        Dx(y) = c(x,y)    /* if y is not a neighbor then c(x,y) = ∞ */
4     for each neighbor w
5        Dw(y) = ? for all destinations y in N
6     for each neighbor w
7        send distance vector Dx = [Dx(y): y in N] to w
8
9  loop
10    wait (until I see a link cost change to some neighbor w or
11          until I receive a distance vector from some neighbor w)
12
13    for each y in N:
14       Dx(y) = minv{c(x,v) + Dv(y)}
15
16    if Dx(y) changed for any destination y
17       send distance vector Dx = [Dx(y): y in N] to all neighbors
18
19 forever
```

Distance Vector (DV) Algorithm

Updating the Forwarding Table

In the DV algorithm, a router updates its distance vector when a link cost changes or it receives new information from a neighboring router.

For each destination, the router identifies the next-hop router that provides the minimum cost. This information is used to update its forwarding table. If multiple neighbors provide the same minimum cost, any one of them can be selected.

Difference Between LS and DV

- Link State (LS): Each router obtains a complete network map and uses Dijkstra's algorithm to find the shortest paths.
- Distance Vector (DV): Each router uses the costs of its directly connected links and information received from its neighbors. It does not require a complete network map.

DV is used in routing protocols such as RIP and BGP.

Working of the DV Algorithm

1. **Initialization**: Each router knows the cost to itself and its directly connected neighbors. Other unknown costs are initially considered infinity.
2. **Exchange:** Each router sends its distance vector to its neighboring routers.
3. **Calculation:** After receiving a neighbor's distance vector, the router calculates the minimum cost to each destination using the Bellman-Ford equation:

$$D_x(y) = \min_v \{c(x,v) + D_v(y)\}$$

4. **Updating:** If the calculated costs change, the router updates its distance vector and forwarding table, then informs its neighbors.
5. **Repetition:** The neighboring routers repeat the same process whenever they receive updated information.

Example

Consider three routers x, y, and z, with link costs:

- x to y = 2
- y to z = 1
- x to z = 7

Initially, router x has the distance vector:

$$D_x = [0, 2, 7]$$

After receiving information from its neighbors, router x recalculates its cost to z:

$$D_x(z) = \min\{2 + 1,\ 7 + 0\}$$

$$D_x(z) = \min\{3, 7\} = 3$$

Therefore, the updated distance vector is:

$$D_x = [0, 2, 3]$$

Router x can reach z at a cost of 3 through router y, instead of using the direct link with a cost of 7.

Convergence

The routers continue exchanging updated distance vectors until no further changes occur. This is called convergence.

Quiescent state

The algorithm reaches a quiescent state when no further distance-vector updates are needed. All routers wait until a link cost changes or new information arrives.

Distance Vector Algorithm: Link-Cost Changes and Link Failure

When a router detects a change in the cost of a directly connected link, it recalculates its distance vector using the Bellman-Ford equation. If its least-cost paths change, it sends the updated distance vector to its neighbors.

When the Link Cost Decreases

Consider a network with three nodes x, y, and z. Suppose the link cost between x and y decreases from 4 to 1.

- At time t0, node y detects the cost change, updates its distance vector, and informs its neighbors.
- At time t1, node z receives the update and recalculates its cost to reach x. Its cost decreases from 5 to 2, so it sends an update.
- At time t2, node y receives the update from z. Since its least cost does not change, it sends no further update.

Thus, the network reaches a stable state quickly. The improved route information spreads rapidly through the network.

When the Link Cost Increases

Now suppose the link cost between x and y increases from 4 to 60, while the link costs $c(y, z) = 1$ and $c(z, x) = 50$ remain unchanged.

Initially:

- $D_y(x) = 4$
- $D_z(x) = 5$

The following events occur:

1. At time t_0, node y detects the increase in link cost and recalculates its route to x:

$$D_y(x) = \min\{60 + 0,\ 1 + 5\} = 6$$

Node y believes it can reach x through z at a cost of 6.

2. At time t_1, node y sends its updated distance vector to z. However, z is still routing to x through y.

3. At time t_2, node z receives the update and recalculates its cost:

$$D_z(x) = \min\{50 + 0,\ 1 + 6\} = 7$$

Node z also believes that the best route is through the other node.

4. The two nodes continue increasing their cost estimates: y calculates 8, z calculates 9, and so on, until z discovers that its direct link to x, with cost 50, is better.

This process creates a **routing loop**, in which packets destined for x travel between y and z instead of reaching x.

Count-to-Infinity Problem

The count-to-infinity problem occurs when routers gradually increase their distance estimates because they mistakenly believe that a neighbor has a valid route to a destination.When a link cost increases or a link fails, this incorrect information can spread slowly through the network, causing routing loops and delayed convergence.

Key point: A decrease in link cost is usually handled quickly, whereas an increase in link cost or a link failure may take much longer to resolve because of the count-to-infinity problem.

Distance-Vector Algorithm: Adding Poisoned Reverse

Poisoned reverse is a technique used in the Distance Vector algorithm to prevent routing loops between two neighboring routers.

Example

Consider three routers x, y, and z, with link costs:

- x-y=4
- y-z=1
- z-x=50

1. **Initially:** Router z reaches x through y, with a total cost of $1 + 4 = 5$.

2. **Poisoned reverse:** Since z uses y to reach x, it advertises the cost to x as infinity (∞) **to** y. This prevents y from mistakenly using z to reach x.

3. **Link cost increases:** Suppose the cost of link $x - y$ increases from 4 to 60. Router y cannot use z, because z advertised infinity to y. Therefore, y keeps its direct route to x, with a cost of 60.

4. **Router z changes its route:** Router z learns that y's cost to x is now 60. It selects its direct link to x, which costs 50, instead of routing through y.

5. **New route:** Router z advertises its actual cost of 50 to y. Router y can now reach x through z, with a total cost of $1 + 50 = 51$.

Limitation: Poisoned reverse prevents routing loops between two neighboring routers, but it cannot prevent all loops involving three or more routers.

Comparison between LS and DV

Feature	Link-State (LS)	Distance-Vector (DV)
Information sharing	Each router shares the costs of its directly connected links with all routers.	Each router shares its least-cost distances to all destinations with its direct neighbors.
Message complexity	Requires O(N × E) messages to distribute link information.	Exchanges messages between neighboring routers during iterations.
Convergence speed	Generally faster; requires $O(N^2)$ operations in the discussed implementation.	May converge slowly and suffer from routing loops and the count-to-infinity problem.
Robustness	Incorrect information mainly affects the link information advertised by that router. Each router calculates its own forwarding table.	Incorrect route information can spread through neighboring routers and affect the entire network.

Routing in the Internet

Internet routing protocols determine the path taken by a datagram from the source to the destination. They use Link-State (LS), Distance-Vector (DV), and Autonomous Systems (AS) concepts.

Autonomous System (AS): An AS is a group of routers managed by the same administration and technical authority. The routers within an AS use a common intra-AS routing protocol, and an AS usually contains multiple subnets.

Routing to an outside destination:

1. The inter-AS protocol identifies the gateways through which the destination subnet can be reached.
2. The intra-AS protocol calculates the least-cost path to each gateway.
3. Hot-potato routing selects the gateway with the lowest internal path cost.
4. The router identifies the outgoing interface and adds the destination and interface to its forwarding table.

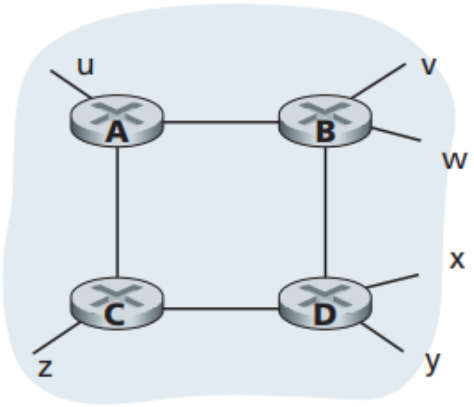
Routing Information Protocol (RIP)

1. Introduction

- RIP is an intra-AS routing protocol, also called an Interior Gateway Protocol (IGP).
- It is a Distance-Vector (DV) protocol used to determine routes within an Autonomous System (AS).
- RIP version 1 is defined in RFC 1058, and RIP version 2 in RFC 2453.

2. Hop Count

- RIP uses hop count as its routing metric. Each subnet traversed adds one hop.
- The hop count includes the destination subnet.
- The maximum valid hop count is 15; a hop count of 16 represents infinity (unreachable). Therefore, RIP is suitable for small networks.



Destination	Hops
u	1
v	2
w	2
x	3
y	3
z	2

Figure 4.34 ♦ Number of hops from source router A to various subnets

3. RIP Routing Table

Each router maintains a routing table containing:

Field	Description
Destination subnet	The subnet to be reached
Next router	The neighboring router used to reach the destination
Number of hops	The hop count along the shortest path

The routing table contains both the router's distance-vector information and forwarding information.

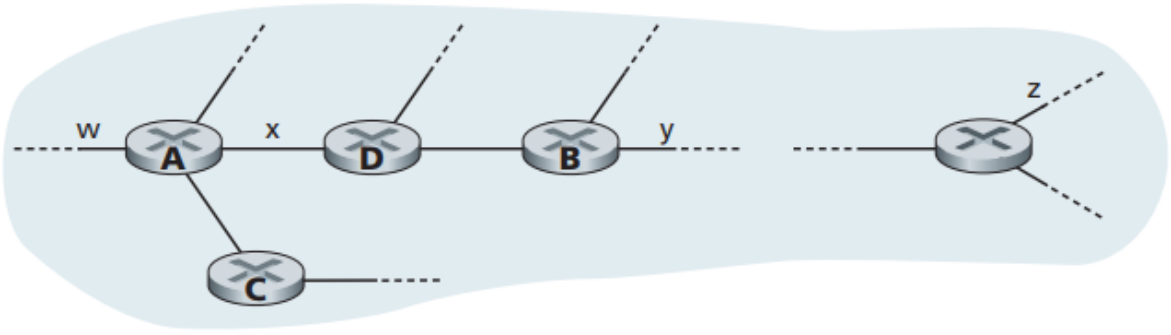


Figure 4.35 ♦ A portion of an autonomous system

Destination Subnet	Next Router	Number of Hops to Destination
w	A	2
y	B	2
z	B	7
x	—	1
....	....	....

Figure 4.36 ♦ Routing table in router *D* before receiving advertisement from router *A*

4. RIP Advertisements and Response Messages

- Neighboring routers exchange routing information approximately every 30 seconds.
- They use RIP response messages, also called RIP advertisements.
- Each response contains a list of up to 25 destination subnets and the sender's distance (hop count) to each subnet.
- A router uses the received advertisement to check whether a better route is available.

Destination Subnet	Next Router	Number of Hops to Destination
z	C	4
w	—	1
x	—	1
....	....	....

Figure 4.37 ♦ Advertisement from router A

5. Updating the Routing Table

1. Router D receives a RIP advertisement from neighboring router A.
2. The advertisement gives A's hop count to different destination subnets.
3. D adds one hop to each advertised distance to calculate the cost through A.
4. D compares the new cost with its existing route.
5. If the new route has a lower hop count, D updates its routing table with the new hop count and next router.

A route may become shorter because the distance-vector algorithm is still converging or because network links or routers have changed.

Destination Subnet	Next Router	Number of Hops to Destination
w	A	2
y	B	2
z	A	5
....	....	....

Figure 4.38 ♦ Routing table in router *D* after receiving advertisement from router *A*

6. Failure Detection and Request Messages

- If a router receives no update from a neighbor for 180 seconds, it considers the neighbor or connecting link unreachable.
- It updates its routing table and sends advertisements to its reachable neighbors.
- RIP also supports request messages, which allow a router to request routing information, such as a neighbor's cost to a destination.

7. RIP Implementation

- RIP sends request and response messages using UDP port 520. These UDP segments are carried inside IP datagrams.
- RIP is implemented as an application-layer process, even though it performs a network-layer routing function.
- In UNIX systems, the process called routed maintains routing information, exchanges messages with neighboring routers, and updates the system's routing tables.

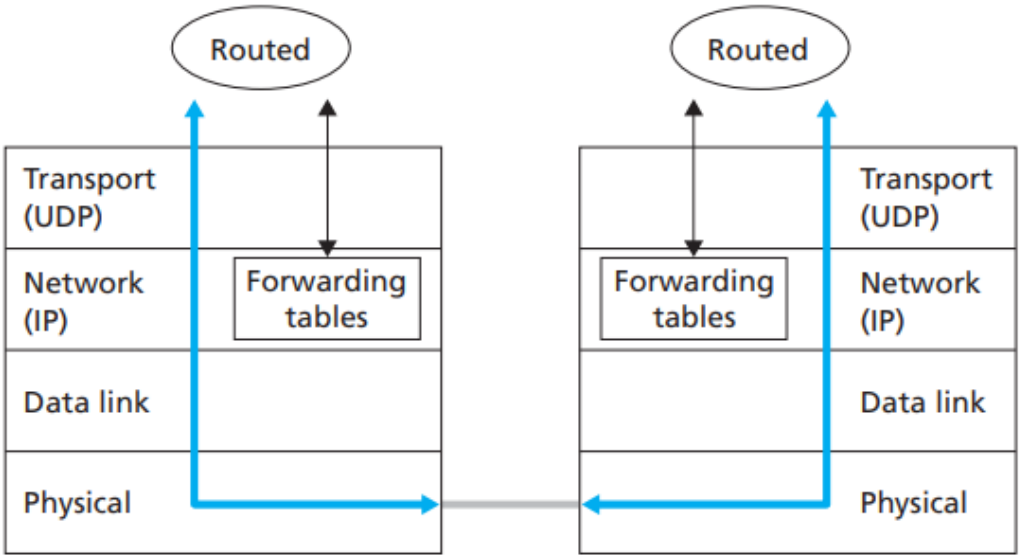


Figure 4.39 ♦ Implementation of RIP as the *routed* daemon

Open Shortest Path First (OSPF)

1. Introduction

- OSPF is an intra-AS routing protocol, also called an Interior Gateway Protocol (IGP).
- It was developed as a successor to RIP and is widely used in upper-tier ISPs.
- OSPF version 2 is defined in RFC 2328.
- OSPF is a Link-State (LS) routing protocol that uses flooding and Dijkstra's least-cost path algorithm.

2. Working of OSPF

1. Each router identifies the state and cost of its directly connected links.
2. It broadcasts link-state information to all routers within the autonomous system using flooding.
3. Each router constructs a complete map of the network topology.
4. It independently runs Dijkstra's algorithm to calculate the least-cost paths to all destinations.
5. The router uses these paths to build its forwarding table.
6. Link-state information is broadcast whenever a link's cost or status changes. It is also broadcast periodically, at least once every 30 minutes.

3. Link Costs

- Link costs are configured by the network administrator.
- Assigning a cost of 1 to every link achieves minimum-hop routing.
- Alternatively, link costs may be set inversely proportional to link capacity, discouraging traffic from using low-bandwidth links.
- OSPF provides the routing mechanism, but the administrator decides the link costs.

4. OSPF Messages

- OSPF advertisements are carried directly by IP using protocol number 89.
- HELLO messages are sent to neighboring routers to check whether they are operational.
- OSPF also allows a router to obtain a neighbor's database of network-wide link-state information.

5. Advanced Features of OSPF

1. Security: Supports authentication to prevent unauthorized routers from introducing incorrect routing information. Simple password and MD5 authentication are supported. MD5 authentication also uses sequence numbers to protect against replay attacks.
2. Multiple same-cost paths: Allows traffic to use multiple paths having the same least cost to a destination.
3. Unicast and multicast routing: Supports unicast routing and multicast routing through Multicast OSPF (MOSPF), which extends the existing link-state mechanism.
4. Hierarchical routing: Allows an autonomous system to be divided into areas, reducing the amount of routing information that each router must handle.

6. Hierarchical OSPF

An OSPF autonomous system can be divided into multiple areas.

- Area: Each area runs its own link-state routing algorithm. Routers broadcast their link-state information to other routers within the same area.
- Area Border Router (ABR): Connects an area to the backbone and handles routing between areas.
- Backbone Area: Exactly one area is designated as the backbone. It connects the other areas and carries inter-area traffic. It contains all ABRs and may also contain other routers.

7. Inter-Area Routing

To send a packet from one area to another:

1. The source router forwards the packet to an ABR in its own area.
2. The packet travels through the backbone area.
3. It reaches an ABR connected to the destination area.
4. The destination-area router forwards the packet to its final destination.